

Senior Developer Technical Exercise

Blast Management API — CQRS & Event Sourcing

Duration: 30 minutes (at home, AI assistance recommended)
Stack: modern C# .NET (8 to 10) — Minimal API or Controllers, your choice
Deliverable: Code pushed to a Git repository (public or shared access)

Context

You are building a small backend slice of a **blast management system** used in the mining industry.
A **blast** groups a set of **holes** drilled into rock. When all holes are charged and ready, the blast is fired. The system must track every state change as an immutable sequence of events — no mutable records, no UPDATE statements.

Domain Model

Blast

Field	Type	Notes
Id	Guid	Unique identifier
Name	string	Human-readable label (e.g. "B-042")
DateBlasted	DateTimeOffset?	Null until the blast fires
Status	enum	Planned → Loaded → Blasted

Hole

Field	Type	Notes
Id	Guid	Unique identifier
BlastId	Guid	Parent blast
Name	string	e.g. "H-01"
Position	(X, Y, Z) numbers	Collar position in local coordinates
Direction	number	Azimuth in degrees (0–360)
Inclination	number	Dip angle in degrees (0 = vertical downward)
Status	enum	Planned → Charged → Ready

What to Build

1. Event Store (in-memory is fine)

Design and implement a simple in-memory event store. It must support:

- **Appending** events for an aggregate (identified by its `Guid`)
- **Loading** the full ordered event stream for an aggregate

You decide the shape of `IDomainEvent`, the stream versioning strategy, and the store interface.

2. Aggregates with Event Sourcing

Implement `Blast` and `Hole` as event-sourced aggregates:

- State is **derived exclusively by replaying events** — no setters, no direct state mutation
- Each aggregate exposes **command methods** (e.g. `blast.Blast(DateTimeOffset now)`) that validate invariants and raise domain events
- Implement at minimum:
 - `CreateBlast` → raises `BlastCreated`
 - `AddHole` → raises `HoleAdded`
 - `ChargeHole` → raises `HoleCharged`; reject if hole is already `Ready` or `Charged`
 - `FireBlast` → raises `BlastFired`; reject if any hole is not in `Ready` or `Charged` status, or if blast is already `Fired`

3. CQRS — Commands & Queries

Implement the following **commands** and **queries** as separate objects (no shared handler, no CRUD controller):

Commands (write side — validate, apply, persist events):

Command	Description
CreateBlastCommand	Create a new blast with a name
AddHoleCommand	Add a hole to an existing blast
ChargeHoleCommand	Mark a hole as charged
FireBlastCommand	Fire the blast, recording the timestamp

Queries (read side — rebuild from events or use a projection):

Query	Returns
GetBlastQuery	Full blast state including all its holes
GetBlastHistoryQuery	Raw ordered list of domain events for a blast

4. HTTP Endpoints

Expose the above via a minimal API or controllers:

POST	/blasts	→ CreateBlastCommand
POST	/blasts/{blastId}/holes	→ AddHoleCommand
PUT	/blasts/{blastId}/holes/{holeId}/charge	→ ChargeHoleCommand
POST	/blasts/{blastId}/fire	→ FireBlastCommand
GET	/blasts/{blastId}	→ GetBlastQuery
GET	/blasts/{blastId}/history	→ GetBlastHistoryQuery

Return meaningful HTTP status codes (400 for domain rule violations, 404 for unknowns, 409 for concurrency conflicts).

Constraints & Rules

- **No database required** — all state lives in-memory
- **No MediatR, no external CQRS framework** — wire it yourself
- **No Aggregate framework** — implement the pattern from scratch

Evaluation Criteria

Your solution will be reviewed against three axes:

Event Sourcing mechanics

- Events are the source of truth; state is never stored directly
- GetBlastHistory returns a clean, typed event log

CQRS pattern understanding

- Commands and queries are clearly separated — different objects, different handlers
- The write side validates business rules before appending events
- The read side rebuilds state by replaying events (no shared mutable state)

Domain modelling accuracy

- Invariants are enforced (FireBlast rejects invalid states)
- Events are named in past tense and capture intent (BlastFired , HoleCharged)
- The domain model is expressive and free of infrastructure concerns

Bonus (if time allows)

- Add a simple **read model / projection** built by subscribing to events, so GetBlast reads from a pre-built view rather than replaying on every query — and explain the trade-off in a code comment
- Add a MarkHoleReady command that transitions a hole from Charged → Ready , and make FireBlast require **all** holes to be Ready (stricter invariant)

Submission

Push your code to a Git repo and share the link. A README is not required, but a brief comment in Program.cs explaining your event store design is appreciated.